

Doing this gives us quick access to a number of useful features of NLTK. Let's start with something simple. Let's see how many times the title of the book is used within its text.

```
print(persuasion.count('persuasion'))
```

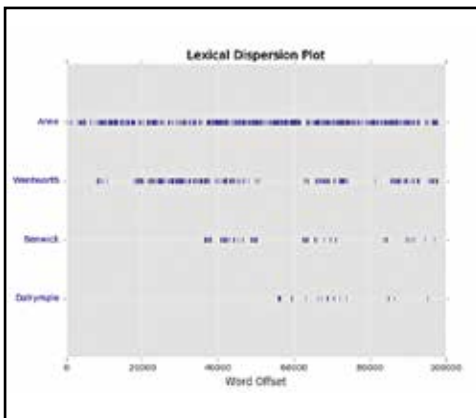
You should get a rather low number of 5. That's interesting. How about we look at each usage of that word in context. For this we use the concordance tool:

```
print(persuasion.concordance('persuasion'))
```

You'll find that the first usage of the word is in the title itself. By default this only lists a total of 79 characters around the word, but you can increase that by providing a width parameter.

If you're writing a book report, or otherwise analysing the text of this book, this can be particularly useful. It would also be useful to analyse the characters in the text.

Let's now look at something called a dispersion plot that shows how a word is distributed in



This graph shows how often the Characters along the y-axis are mentioned over the course of the book.

the text. Let's see how four different characters appear throughout the book.

```
persuasion.dispersion_plot(["Anne", "Wentworth", "Benwick", "Dalrymple"])
```

Looking at the graph you can see how references to these characters are distributed throughout the book. You can tell from the graph that "Anne" is an important character, since she's mentioned a lot – she's the protagonist. You can see that "Benwick" is introduced a third of the way through the book, and is central in a few areas but mostly fades out. "Lady Dalrymple" is introduced mid-way through the book and also fades out.

*pointers

>>Interesting PHP developer videos



*Introduction to NLP

>>A playlist from that introduces the concepts involved in NLP, by professors from University of Stanford.

<http://dgit.in/IntroNLP>



*NLP in Python

>>Tutorial on NLP with Python and Natural Language Tool Kit, a python suite of libraries and programming for NLP.

<http://dgit.in/NLPpython>



*Machines that learn

>>A TED talk on the implications of machines getting natural language processing capabilities by Jeremy Howard.

<http://dgit.in/MLJeremy>



*Google Assistant

>>Google announced Google Assistant, an NLP enabled smart home assistant that will enable interactive home appliance control

<http://dgit.in/GgleAstrnt>



What we've done till now has been great, but it doesn't really need any understanding of the language by the computer to work. Let's look at a simple concept called stemming.

Stemming

Language – especially the English language – naturally contains many affixed forms for many words. For example, "coding", "coded", and "codes" all stem from the word "code".

It can be of great help to break a word down to its core meaning, to its "stem word" as it's called. You can imagine how useful this would be in a search engine where searching for "coding" should also return results for mentions of "code", and "codes".

Both the above examples should resolve to "walk".

NLTK includes a number of stemming algorithms to get to the stem word for any given word. We'll look at the Porter stemmer, which isn't the best but is easy to use and fast.

```
ps = nltk.PorterStemmer()
print(ps.stem('walking'))
print(ps.stem('walked'))
```

There is a more powerful concept called lemmatizing that can go even further, by reducing words based on similar meaning. If you lemmatize "are", and "is" as verbs they will both resolve to "be". You can try the WordNet Lemmatizer using `nltk.WordNetLemmatizer()`.

Parts of Speech

You might be familiar with Parts of Speech which classify words as nouns, verbs, adjectives, etc. It could be of immense use in some cases to be able to derive the class of all the words in a sentence.

Let's see how we can have NLTK tag the parts of speech for all the words in a sentence.

```
sentence = nltk.word_tokenize('Anne hoped she had outlived the age of blushing; but the age of emotion she certainly had not.')
print(nltk.pos_tag(sentence))
```

You'll notice that we are first splitting the sentence into words, and then using the `pos_tag` function on this sentence as a list of words. You cannot directly use `pos_tag` without breaking the sentence into a word list.